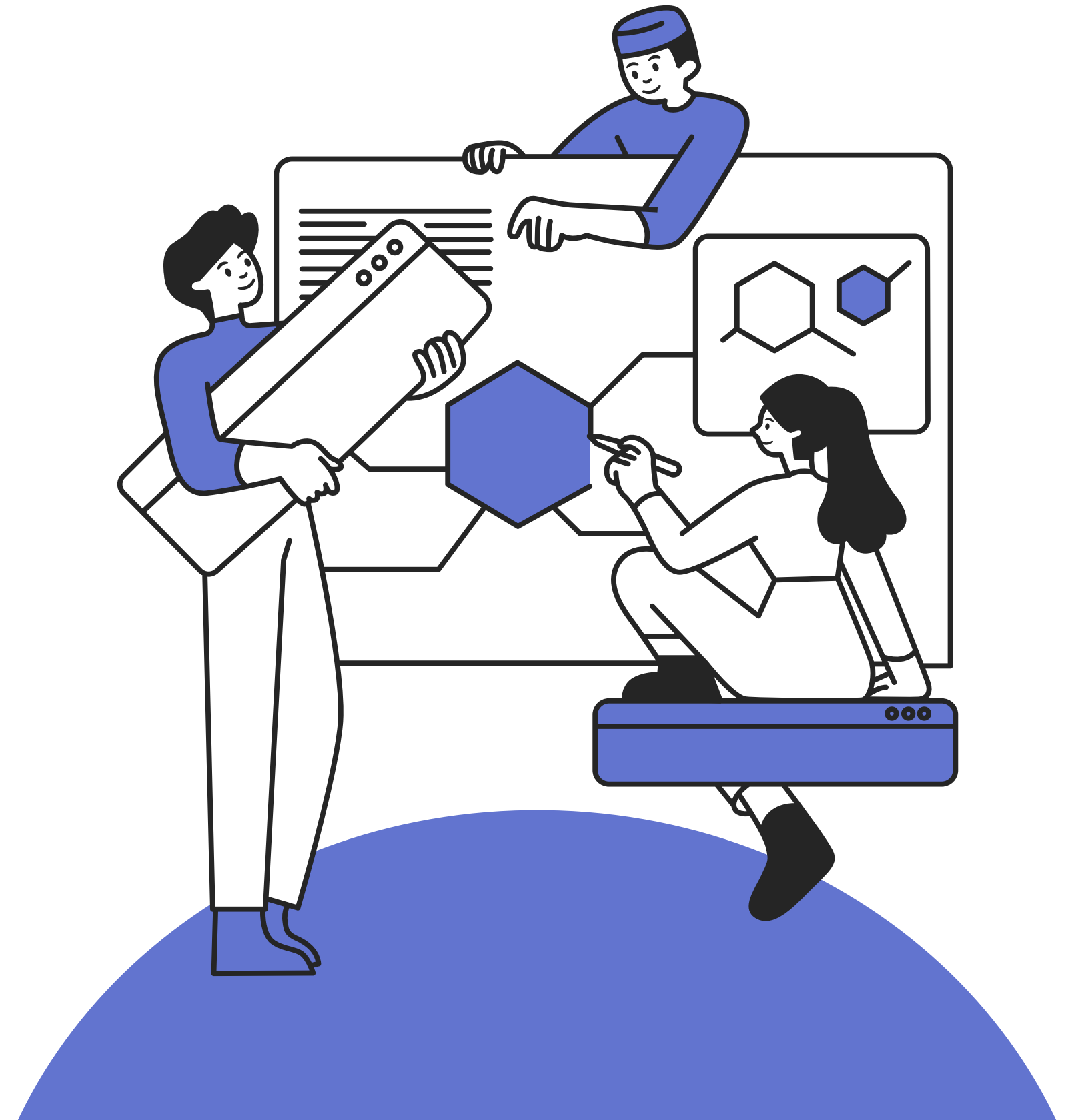


알고리즘 설계와 성능 분석

이예린T



학생1: 문제1번 해결하는 데 며칠 걸릴 것 같아?

학생: 한.. 이틀?

48시간 걸린다?

1.1일 걸린다?



CONTENTS

- 1 알고리즘 성능 분석
- 2 Brute force 알고리즘
- 3 모듈별 탐구
- 4 알고리즘 설계와 성능 분석 정리

01

알고리즘

알고리즘이란?

- 주어진 문제를 해결하기 위한 단계적인 절차
- 특정한 일을 수행하는 명령어들의 집합
- 프로그래밍 언어와 상관없이 문제 해결 절차를 나타내는 명령어의 집합
- 알고리즘의 조건



▲ [그림 Ⅲ-6] 알고리즘의 조건

01

알고리즘

알고리즘이란?

- 알고리즘의 성능 측정 기준
 - 정확성: 정확하게 동작하는가?
 - **연산량**: 얼마나 적은 연산을 수행하는가? → 시간 효율성
 - **메모리 사용량**: 얼마나 적은 메모리를 사용하는가? → 공간 효율성
 - 단순성: 얼마나 단순한가?
 - 최적성: 더 이상 개선할 여지가 없을 만큼 최적화되어 있는가?

02

알고리즘의 성능 분석

복잡도 분석

- `time.time()`을 사용하면 실제 실행 시간을 측정할 수 있음.
 - 알고리즘을 반드시 구현하여 확인해야 함.
 - 같은 조건의 하드웨어를 사용해야 비교 가능
 - 같은 조건의 소프트웨어 환경이어야 비교 가능
 - 실험되지 않은 입력에 대해서는 실행 시간 주장 불가(입력 값에 따라 전혀 다른 결과가 나올 수 있다).
- 따라서 복잡도 분석을 통해 알고리즘의 성능을 비교한다.
 - 시간 복잡도, 공간 복잡도
- n 의 크기가 커질수록 알고리즘의 성능 차이는 극단적으로 벌어진다.
- n 의 크기가 커지면 커질수록 최고차 항을 제외한 나머지 항의 영향은 거의 없다.

N	$n^2 + 5n$	$27n$
1	6	27
10	150	270
100	10,500	2,700
1,000	1,005,000	27,000
10,000	100,050,000	270,000
100,000	10,000,500,000	2,700,000
1,000,000	1,000,005,000,000	27,000,000
10,000,000	100,000,050,000,000	270,000,000
100,000,000	10,000,000,500,000,000	2,700,000,000
1,000,000,000	1,000,000,005,000,000,000	27,000,000,000

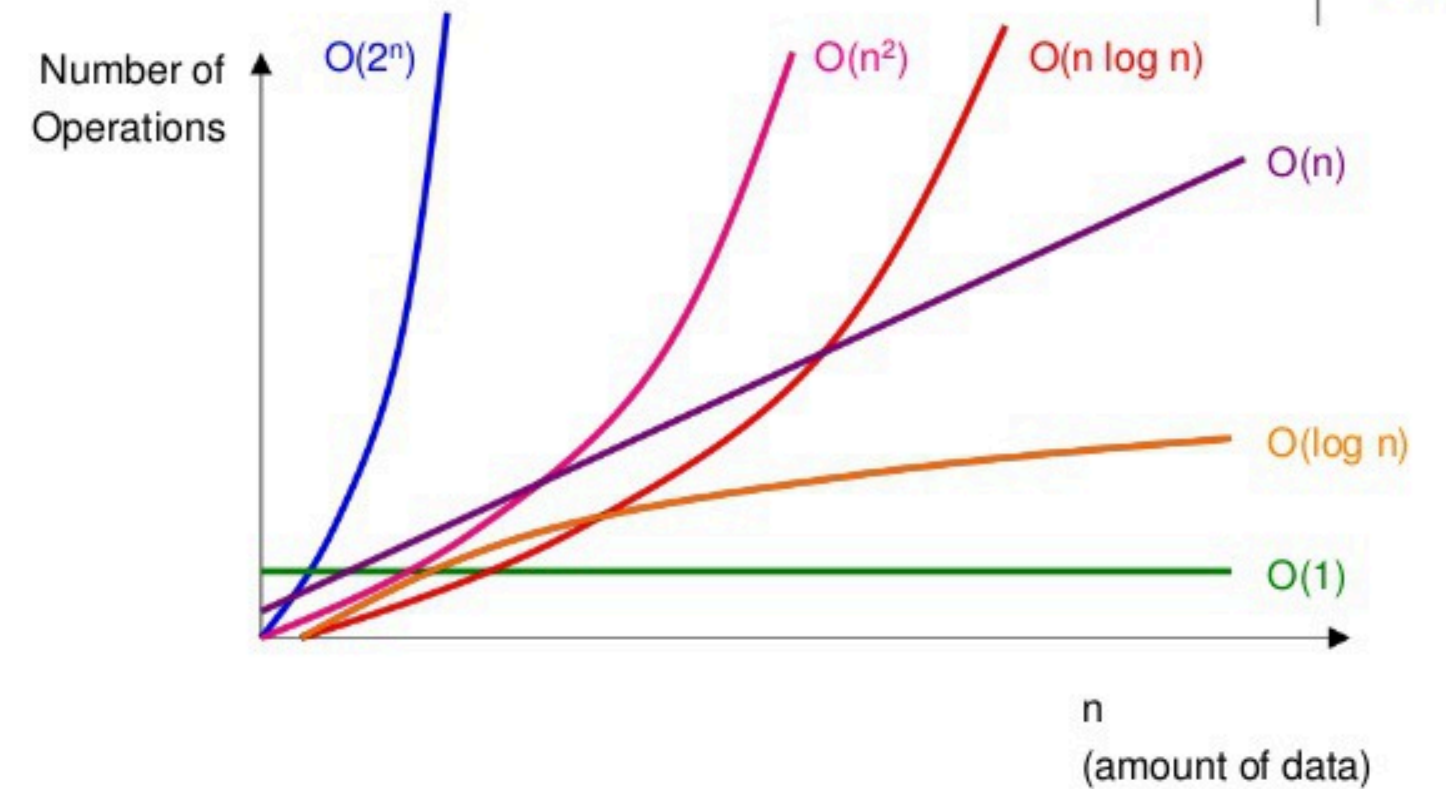
03

점근 표기법

대략적인 성능 표기

- n 의 크기가 커지면 커질수록 최고차 항을 제외한 나머지 항의 영향은 거의 없다.
- 따라서 점근 표기법에서는 알고리즘의 성능을 **최고차 항으로만** ‘대략적으로’ 표시한다.
- 대표적인 점근 표기법
 - O (Big O) 표기법: 최악의 경우 알고리즘 수행 시간
 - Ω (Big Omega) 표기법: 수행해야 하는 최소한의 수행 시간
 - Θ (Big Theta) 표기법: 수행 시간의 상한이자 하한

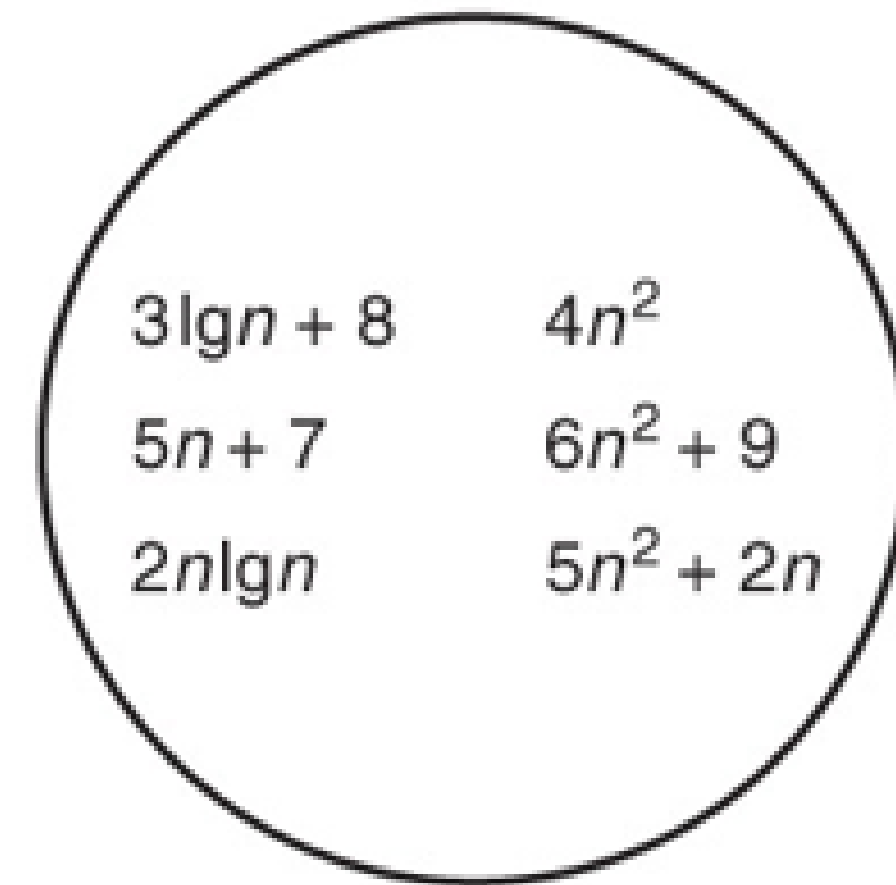
Comparing Big O Functions



04

$O(\text{Big-O})$ 표기법

- $O(g(n))$: 증가속도가 $g(n)$ 과 같거나 낮은 모든 복잡도 함수를 포함하는 집합
- 어떤 알고리즘의 복잡도가 $O(n)$ 이라면, 그 알고리즘은 n 에 비례하는 시간 내에 반드시 완료된다.
- n 보다 더 빨리 처리될 수는 있지만, 그보다 더 걸리진 않는다.
- 즉, 처리 시간의 상한이다.

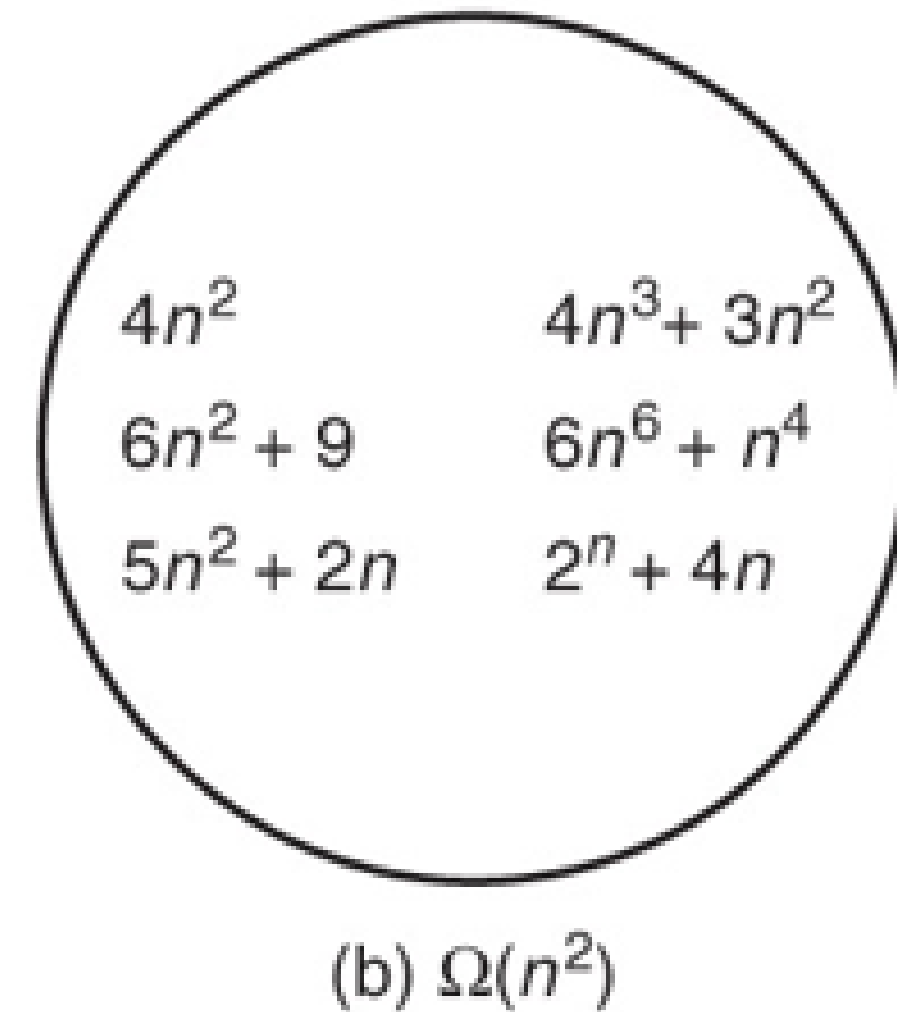


(a) $O(n^2)$

05

Ω (Big-Omega) 표기법

- $\Omega(g(n))$: 증가속도가 $g(n)$ 과 같거나 높은 모든 복잡도 함수를 포함하는 집합
- 어떤 알고리즘의 복잡도가 $\Omega(n)$ 이라면, 그 알고리즘은 아무리 빨리 처리하더라도 n 에 비례하는 시간 이상은 반드시 걸린다.
- 즉, 처리 시간의 하한이다.



(b) $\Omega(n^2)$

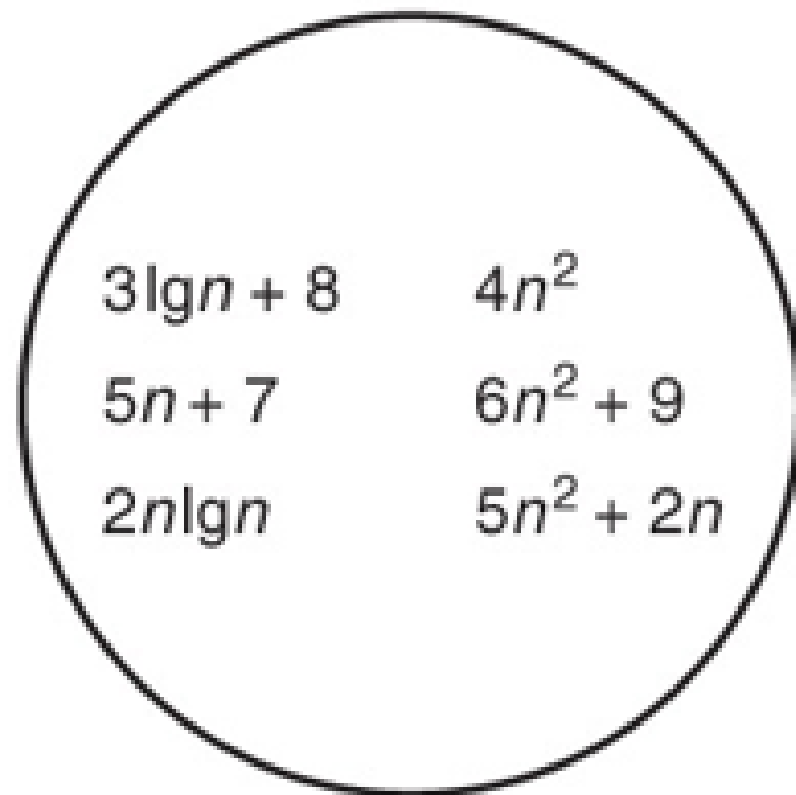
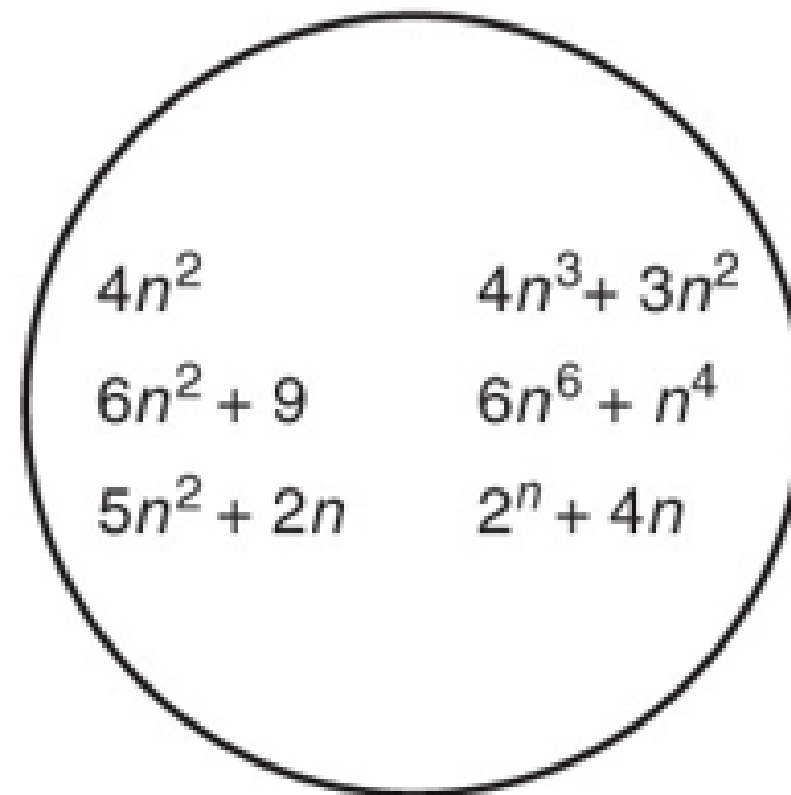
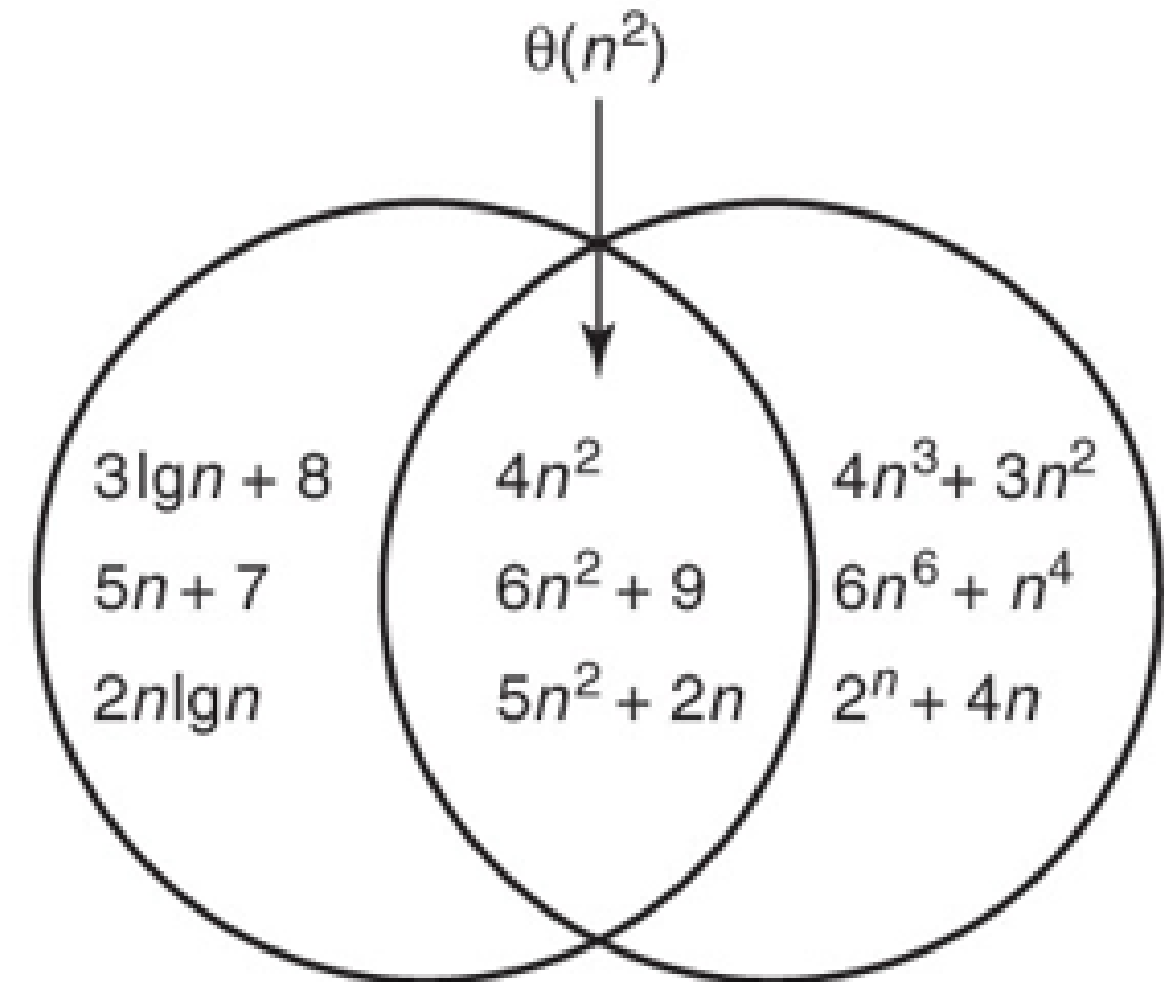
The circle contains the following expressions:

$4n^2$	$4n^3 + 3n^2$
$6n^2 + 9$	$6n^6 + n^4$
$5n^2 + 2n$	$2^n + 4n$

06

 Θ (Big-Theta) 표기법

- $\Theta(g(n))$: 증가속도가 $g(n)$ 과 같은 복잡도 함수들만을 포함.
- 즉, 처리 시간의 상한인 동시에 하한인 경우

(a) $O(n^2)$ (b) $\Omega(n^2)$ (c) $\Theta(n^2) = O(n^2) \cap \Omega(n^2)$

07

어느 표기법을 사용할까?

O(빅-오)

- 시간 복잡도를 정확히 계산할 수 있다면 빅세타 표기법을 사용
- 정확히 분석하기 어렵다면 상한을 구해 빅오 표기법 적용
- 일반적으로는 최악의 상황을 고려한 해결책을 찾기 때문에 빅오 표기법이 주로 사용된다.
- 빅오 표기의 수행 시간을 순서대로 나열 →

- $O(1)$: 상수형
- $O(\log n)$: 로그형
- $O(n)$: 선형
- $O(n \log n)$: 로그선형
- $O(n^2)$: 2차형
- $O(n^3)$: 3차형
- $O(n^k)$: k차형
- $O(2^n)$: 지수형
- $O(n!)$: 팩토리얼형

08

O(빅-오) 계산하기

- 시간 복잡도를 정확히 계산할 수 있다면 빅세타 표기법을 사용
- 정확히 분석하기 어렵다면 상한을 구해 빅오 표기법 적용
- 일반적으로는 최악의 상황을 고려한 해결책을 찾기 때문에 빅오 표기법이 주로 사용된다.
- 빅오 표기의 수행 시간을 순서대로 나열 →
 - $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(3^n) < O(n!)$

08

O(빅-오) 계산하기

```
i = 1;
while (i <= n) {
  x = x + 1;
  i = i + 1;
}
```

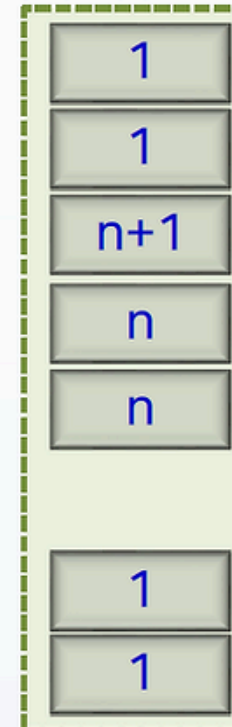
$3n+2 \rightarrow O(n)$

```
int i, j;
for (i=1; i<=n; i++)
  for (j=i; j<=n; j++)
    if (j > i)
      C[i,j] = A[i]*B[j];
```

$O(n^2)$

```
SumAverage( A[], n)
{ //A[0..n-1], n : 입력 배열과 데이터 개수
1  sum = 0;
2  i = 0;
3  while ( i < n ) {
4    sum = sum + A[i];
5    i = i + 1;
6  }
7  average = sum / n;
8  print sum, average;
9 }
```

n : 입력 크기



$f(n) = 3n+5$

점근성능

$O(n)$

Big-Oh
표기

- 1부터 n까지 합 구하기
 - 값 증가시키며 누적하여 더하기 $\rightarrow O(T(n)) = O(2n+1) = O(n)$
 - 공식 이용해 더하기 $\rightarrow O(T(n)) = O(4) = O(1)$

```
sum = 0
for i in range(n):
  sum += n
return sum
```

```
sum = n*(n+1) / 2
return sum
```


08

O(빅-오) 계산하기

재귀함수

```
int RecurrenceSum(int Data[], int SizeOfData) {  
    if(SizeOfData == 1)  
        return Data[0];  
    else  
        return RecurrenceSum(&Data[1], SizeOfData-1) + Data[0];  
}
```

```
factorial(n)  
{  
    if(n == 1) return 1;  
    return n*factorial(n-1);  
}
```

08

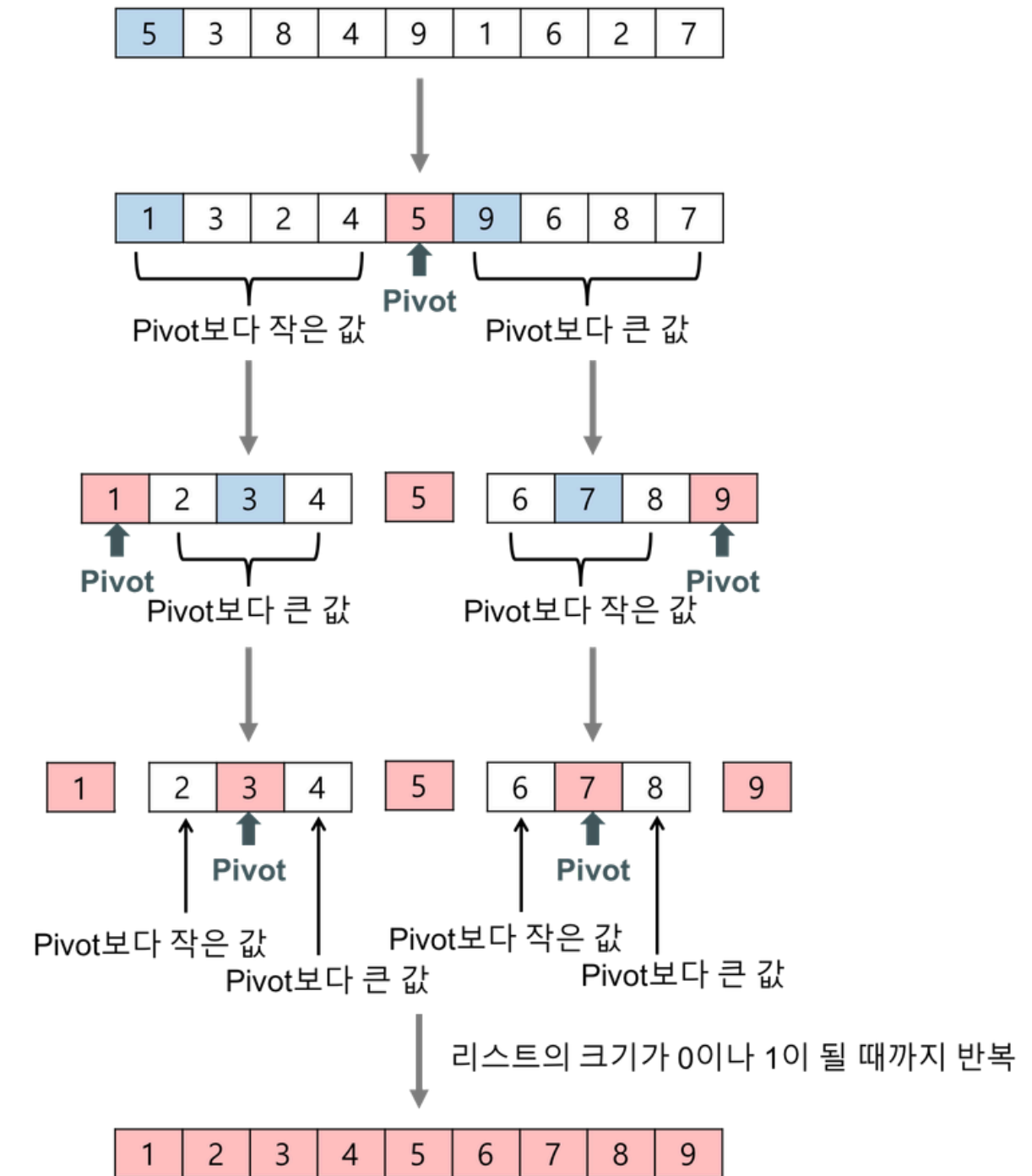
O(빅-오) 계산하기

퀵 정렬

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    lesser_arr, equal_arr, greater_arr = [], [], []
    for num in arr:
        if num < pivot:
            lesser_arr.append(num)
        elif num > pivot:
            greater_arr.append(num)
        else:
            equal_arr.append(num)
    return quick_sort(lesser_arr) + equal_arr + quick_sort(greater_arr)
```

초기상태

5	3	8	4	9	1	6	2	7
---	---	---	---	---	---	---	---	---

오름차순
완성상태

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

08

O(빅-오) 계산하기

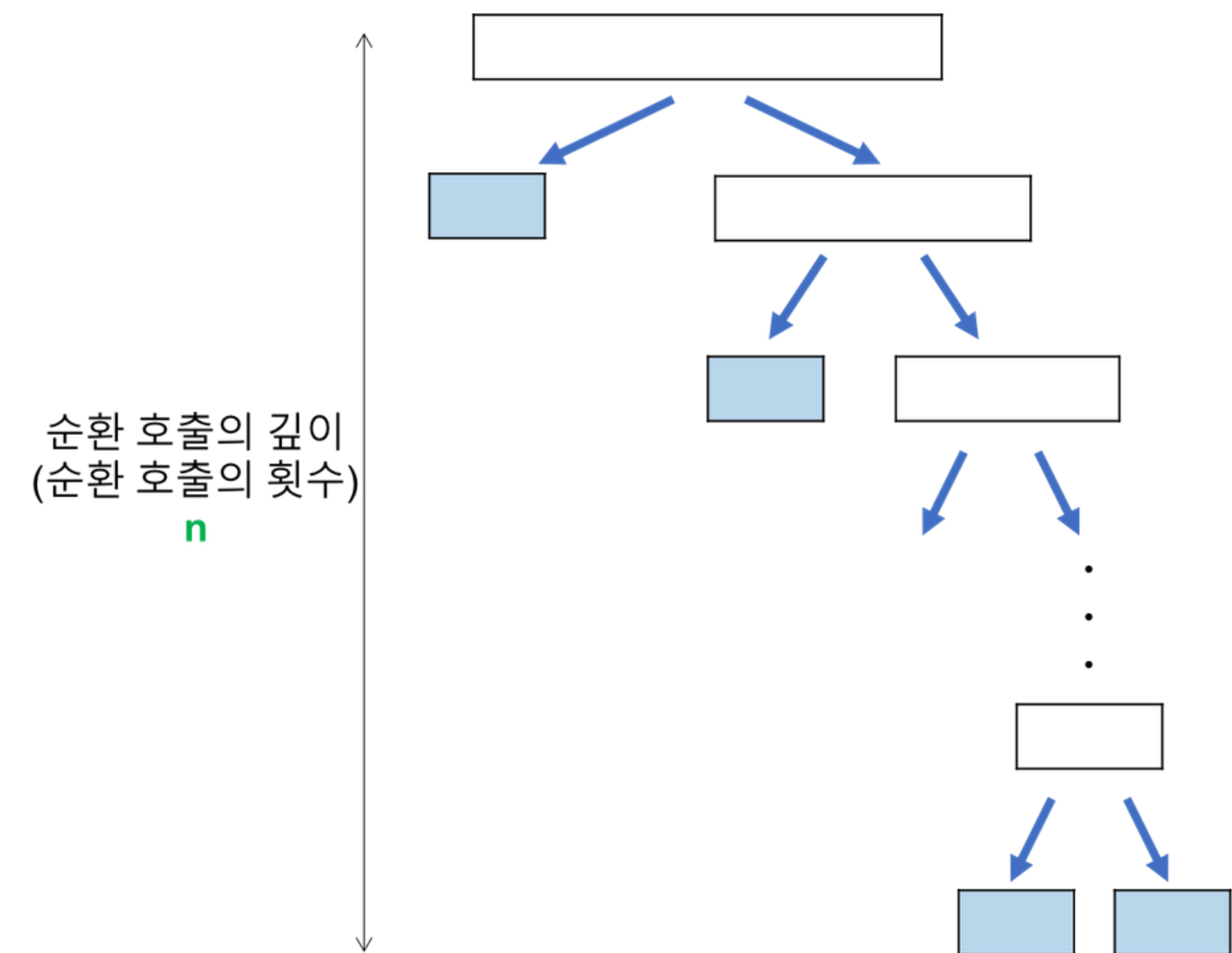
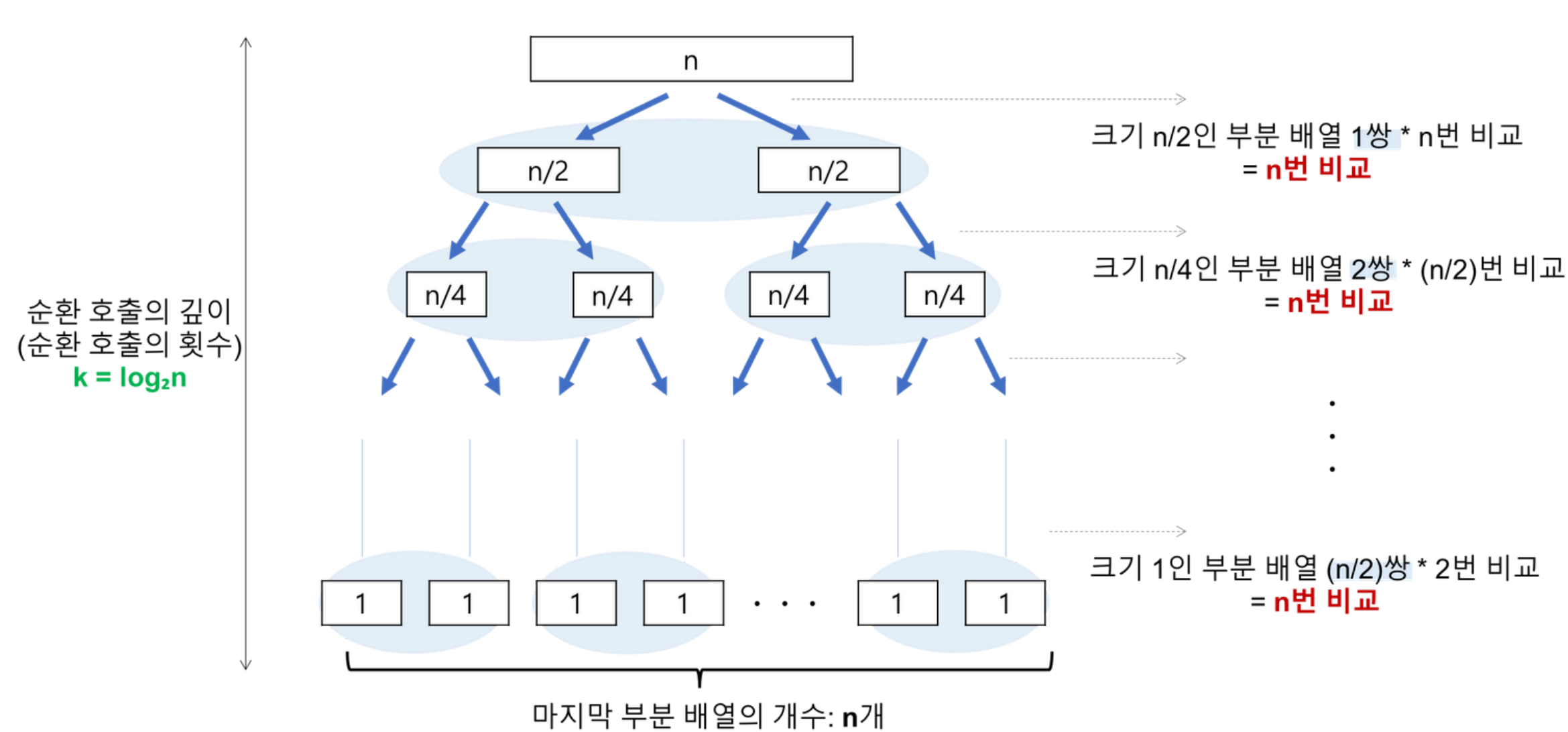
퀵 정렬

```
def quick_sort(array, start, end):  
    if start >= end:  
        return  
    pivot = start #피벗 초기값은 첫번째 요소  
    left = start+1  
    right = end  
  
    while left <= right:  
        while left <= end and array[left] <= array[pivot]:  
            left+=1  
        while right > start and array[right] >= array[pivot]:  
            right-=1  
        if left>right: # 엇갈렸다면 작은 right -=1 데이터와 피벗을 교체  
            array[right], array[pivot] = array[pivot], array[right]  
        else: # 엇갈리지 않았다면 작은 데이터와 큰 데이터를 교체  
            array[left], array[right] = array[right], array[left]  
    quick_sort(array, start, right-1)  
    quick_sort(array, right+1, end)  
  
quick_sort(array, 0, len(array)-1)
```

08

O(빅-오) 계산하기

퀵 정렬



01

Brute Force 알고리즘

- brute: 무식한
- force: 힘
- 해가 존재할 것으로 예상되는 모든 영역을 전체 탐색한다.
- 예외 없이 100%의 확률로 정답을 구할 수 있다.

01

시간복잡도 구하기

- 각 정렬 알고리즘의 시간복잡도: 삽입 정렬, 선택 정렬, 병합 정렬, 퀵 정렬, 버블 정렬
 $n, n^2 \quad n^2 \quad n \log n \quad n \log n, n^2 \quad n^2$
- 각 탐색 알고리즘의 시간복잡도: 순차 탐색, 이진 탐색
 $n \quad \log n$
- 각각의 시간복잡도
 - 우선순위 큐(힙) - 삽입, 삭제, 특정 값 탐색
 $\log n \log n \quad n$
 - 해시 테이블 - 삽입, 삭제, 특정 값 탐색 / 모두 1
 - 최단 경로 탐색 - 다익스트라, 벨만포드
 $n^2 \quad n^3$

02

합이 k인 두 수 만들기

정수 배열 nums와 정수 k가 주어질 때, 합이 k가 되는 두 수의 인덱스를 반환하라.

- 1) brute force로 해결하라.
- 2) 시간 복잡도는 $O(n)$ 이하로 해결하라.
- 3) 두 코드의 실행 시간을 비교하라.

03

성적 상위 k명 뽑기

n명의 학생 성적이 담긴 배열 scores가 주어질 때, 상위 k명의 성적을 내림차순으로 반환하라.

- 1) brute force로 해결하라.
- 2) 시간 복잡도 $O(n \log k)$ 이하로 해결하라.
- 3) 두 방법의 실행 시간을 비교하라.

04

계단 오르기

한 번에 1칸 또는 2칸씩 오를 수 있다.

n 이 입력될 때, n 번째 계단까지 오르는 방법의 수를 출력하라.

(단, 시간 복잡도는 $O(n)$ 이하로 한다.)

05

가장 가까운 두 수

정수 n 개를 입력 받는다.

서로 다른 두 수의 차이 중 가장 작은 값을 구하시오.

(단, 시간 복잡도는 $O(n \log n)$ 이하로 한다.)

06

섬의 개수 세기

1(땅)과 0(물)로 이루어진 $m \times n$ 격자를 입력받는다.

상하좌우로 연결된 땅 덩어리를 섬이라 할 때, 전체 섬의 개수를 반환하라.

(단, 시간 복잡도는 $O(mn)$ 이하로 한다.)

Q&A
